

Challenge Write-Up Template

Instructions: Copy this template for every challenge you submit. Fill in every section — a write-up that only shows the final flag without explanation will not receive full credit. Delete these italicized instruction lines before submitting.

Challenge Information

Student Name	<i>Facon, Janna Mae L.</i>
Section / Class	<i>BSIT- 3G</i>
Challenge Name	<i>Password Profiler</i>
Category	<i>General Skills</i>
Points / Difficulty	<i>Easy</i>
Date Solved	<i>8/13/2026</i>
Flag Format Given	<i>PicoCTF{...}</i>

1. Challenge Description

Restate the challenge in your own words: what were you given, and what were you asked to find or do? Do not simply copy-paste the original prompt.

The Password Profiler challenge involved recovering a password from a SHA-1 hash. The challenge provided personal information about the target, a file containing the password hash, and a Python script for checking possible passwords against the hash. The goal was to use the available information to create or use a customized password list, find the password that matched the SHA-1 hash, and then use the recovered password to obtain the flag.

2. Initial Analysis / Reconnaissance

What did you check first? What did file, strings, a webpage view-source, or another first look tell you? What category did you decide this was, and why?

The first step I did was read the challenge description carefully. I identified that this was a **General Skills** challenge involving password recovery and OSINT-based password generation. The challenge provided three important files: `userinfo.txt`, which contained personal details about the target; `hash.txt`, which contained the SHA-1 hash; and `check_password.py`, which was the script used to test passwords against the hash. I then inspected the files in the terminal using `ls` and saw the available files, including `alice.txt`, `check_password.py`, `cupp.cfg`, `cupp.py`, `hash.txt`, and `userinfo.txt`. I also checked the available password information and found a list of possible passwords containing variations of the target's name, such as `johnsonAj_5`, `johnsonAj_7`, and other similar combinations.

3. Tools and Commands Used

List every tool/command you used, in the order you used them, and what each one told you.

Tool / Command	What it was used for	Result / Output
ls	Checked the files available in the challenge directory.	Found alice.txt, check_password.py, cupp.cfg, cupp.py, hash.txt, and userinfo.txt.
less alice.txt	Examined the generated password list.	Displayed many possible password combinations based on the target's information.
python check_password.py	Ran the provided password-checking script.	Initially returned an error because passwords.txt was not found.
Password list generation	Created the password list required by the checking script using the provided target information.	Generated candidate passwords for testing against the SHA-1 hash.
python check_password.py	Tested the generated passwords against the target hash.	Successfully found the password: picoCTF{Aj_15901990}.
echo -n 'Aj_15901990' sha1sum	Verified the SHA-1 hash of the recovered password.	Produced 968c23490427d5d7debe72385a2c00eca5, matching the hash provided by the challenge.

4. Solution Walkthrough

Explain your reasoning step by step — not just what you typed, but why you tried it and what you concluded from the result. Include screenshots, code snippets, or command output where useful. A reader who has not seen the challenge should be able to follow your logic and arrive at the same answer.

Step 1

The first step I did was read the challenge description to understand what I needed to find. I learned that the challenge provided personal information about a target, a SHA-1 password hash, and a script called check_password.py that could be used to test possible passwords. I then used ls in the terminal to see the available files. The directory contained userinfo.txt, hash.txt, check_password.py, cupp.py, cupp.cfg, and other files. I understood that the personal information

Step 2

I examined the available password candidates and saw many variations based on the target's information, including combinations such as johnsonAj_5, johnsonAj_7, and similar entries. I then ran the provided checking script with: `python check_password.py` The first time I ran it, the script returned an error indicating that passwords.txt could not be found. This showed me that the script expected a password list named passwords.txt. After creating the required password list from the available target information, I ran the checking script again. This time, the script successfully tested the candidates and displayed: `Password found: picoCTF{Aj_15901990}` The recovered password was therefore Aj_15901990.

I examined the available password candidates and saw many variations based on the target's information, including combinations such as johnsonAj_5, johnsonAj_7, and similar entries. I then ran the provided checking script with: `python check_password.py` The first time I ran it, the script returned an error indicating that passwords.txt could not be found. This showed me that the script expected a password list named passwords.txt. After creating the required password list from the available target information, I ran the checking script again. This time, the script successfully tested the candidates and displayed: Password found: picoCTF{Aj_15901990} The recovered password was therefore Aj_15901990.

[illegible]

Step 3

To verify that the recovered password actually matched the SHA-1 hash provided by the challenge, I used: `echo -n 'Aj_15901990' | sha1sum` The terminal returned: `968c23490427d5d7debe72385a2c00eca5` This matched the SHA-1 hash in `hash.txt`, confirming that `Aj_15901990` was the correct password. I then used the recovered password/flag shown by the checking script and entered `picoCTF{Aj_15901990}` into the **FLAG** field on the challenge page.



5. Flag

picoCTF{Aj_15901990}

6. Key Concept / What I Learned

In 2-4 sentences, name the underlying security concept or vulnerability class this challenge taught (e.g. "weak RSA exponent," "reflected XSS," "stack buffer overflow"). This is the part of the write-up most worth remembering after the flag is submitted.

*This challenge taught me about **password profiling and targeted password-list generation**. I learned that personal information can be used to generate likely password combinations, which can make password attacks much more effective than using a completely random wordlist. I also learned how a known SHA-1 hash can be used to verify whether a recovered password is correct.*

7. References

List any external resource you consulted beyond course materials (a tool's documentation, a technique explainer, a similar public write-up). It is normal and expected to use references — just cite them.

https://youtu.be/_Qqe9tmHHeI?si=zwI2JZ-PHPjiBf27
